Experiment No. 8
To implement a Deep Q-Network (DQN) for Atari Game Playing.
Date of Performance:
Date of Submission:
Marks:
Sign:

Aim: Deep Reinforcement Learning: Implementing a deep Q-network (DQN) to train an agent to play a popular Atari game

Objective: the goal is to develop an AI agent capable of autonomously learning and executing effective strategies to achieve high scores and proficient gameplay in complex Atari game environments.

Theory:

Implementing a Deep Q-Network (DQN) to train an agent to play Atari games like Pong or Space Invaders is a popular application of reinforcement learning. Here's a high-level overview of how you can do this:

1. Environment Setup:
    - Install the necessary libraries like Gym Atari, which provides an interface to Atari games.
    - Set up your Python environment with libraries like TensorFlow or PyTorch for deep learning.
2. Understanding the Environment:
    - Choose an Atari game (e.g., Pong, Space Invaders) and understand its state space, action space, and reward structure.
3. Deep Q-Network (DQN) Implementation:
    - Define a neural network architecture for your DQN. Typically, a convolutional neural network (CNN) is used to process game frames.
    - Implement the Q-network, which takes the game state as input and outputs Q-values for each action.
    - Implement an experience replay buffer to store past experiences (state, action, reward, next state).
    - Implement the epsilon-greedy strategy for action selection to balance exploration and exploitation.
    - Define the loss function, typically the mean squared error between predicted Q-values and target Q-values.
    - Train the DQN by sampling batches of experiences from the replay buffer and updating the Q-network parameters using gradient descent.
4. Training Process:

- Initialize the DQN with random weights.
- Interact with the environment using the current policy (epsilon-greedy) and collect experiences.
- Store experiences (state, action, reward, next state) in the replay buffer.
- Sample batches of experiences from the replay buffer and perform a gradient descent step to update the Q-network parameters.
- Update the target Q-network periodically to stabilize training.

5. Evaluation:
    - Periodically evaluate the performance of the trained agent by running it in the environment and measuring its average reward or other performance metrics.
    - Adjust hyperparameters and tweak the algorithm as needed based on performance.
6. Fine-tuning:
    - Experiment with different hyperparameters, neural network architectures, and training strategies to improve the agent's performance.
    - Consider advanced techniques like double DQN, dueling DQN, prioritized experience replay, or rainbow DQN for further improvements.
7. Deployment:
    - Once the agent achieves satisfactory performance, deploy it to play the game autonomously or integrate it into a larger system.

Remember that training DQN can be computationally intensive and may require significant time and computational resources, especially for complex Atari games. Be patient and monitor the training process closely to diagnose any issues and make necessary adjustments. Additionally, consider leveraging cloud computing resources or distributed training techniques to speed up the training process.

Implementation:

```
import gym
import numpy as np
import tensorflow as tf
```

```
from tensorflow.keras import layers
import random
from collections import deque

# Step 1: Load Environment
env = gym.make("CartPole-v1")  # simpler alternative to Atari for lab

state_size = env.observation_space.shape[0]
action_size = env.action_space.n

# Step 2: Hyperparameters
gamma = 0.95
epsilon = 1.0
epsilon_min = 0.01
epsilon_decay = 0.995
learning_rate = 0.001
batch_size = 32
memory = deque(maxlen=2000)

# Step 3: Build DQN Model
def build_model():
    model = tf.keras.Sequential([
        layers.Dense(24, activation='relu', input_shape=(state_size,)),
        layers.Dense(24, activation='relu'),
        layers.Dense(action_size, activation='linear')
    ])
    model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=learning_rate),
                  loss='mse')
    return model

model = build_model()
```

```
# Step 4: Store Experience
def remember(state, action, reward, next_state, done):
    memory.append((state, action, reward, next_state, done))

# Step 5: Choose Action
def act(state):
    global epsilon
    if np.random.rand() <= epsilon:
        return random.randrange(action_size)
    q_values = model.predict(state, verbose=0)
    return np.argmax(q_values[0])

# Step 6: Replay (Training)
def replay():
    global epsilon
    if len(memory) < batch_size:
        return

    minibatch = random.sample(memory, batch_size)

    for state, action, reward, next_state, done in minibatch:
        target = reward
        if not done:
            target += gamma * np.amax(model.predict(next_state, verbose=0)[0])

        target_f = model.predict(state, verbose=0)
        target_f[0][action] = target

        model.fit(state, target_f, epochs=1, verbose=0)

    if epsilon > epsilon_min:
        epsilon *= epsilon_decay
```

```
# Step 7: Training Loop
episodes = 100

for e in range(episodes):
    state = env.reset()[0]
    state = np.reshape(state, [1, state_size])

    for time in range(500):
        action = act(state)
        next_state, reward, done, _, _ = env.step(action)

        next_state = np.reshape(next_state, [1, state_size])
        remember(state, action, reward, next_state, done)

        state = next_state
        replay()

        if done:
            print(f"Episode: {e+1}/{episodes}, Score: {time}, Epsilon: {epsilon:.2f}")
            break
```

`

Conclusion:

1. What is DQN and how is it different from Q-learning?

- **DQN (Deep Q-Network)** uses a neural network to approximate Q-values instead of a Q-table.
- Traditional Q-learning uses a table, which is not scalable for large state spaces.
- DQN can handle high-dimensional inputs like images (Atari frames).

2. Challenges in Training DQN & Solutions:

Challenge	Solution
Unstable learning	Use target network
Correlated data	Use experience replay
Overestimation	Use Double DQN
Slow convergence	Tune hyperparameters
Large state space	Use CNNs